Unit 5
Spring Boot Framework-Part-2

Spring Boot Framework-Part-2: The Data Layer:

- The spring boot follows a **layered architecture** in which each layer communicates to other layers(Above or below in hierarchical order). The spring boot documentation provides the following definition to the Spring Boot Framework.
- **Database Layer**
- The database layer contains all the databases such as MySql, MongoDB, etc. This layer can contain multiple databases. It is responsible for performing the CRUD operations.

Database Layer

- **CRUD** stands for **Create, Read/Retrieve, Update, and Delete** and these are the four basic operations that we perform on persistence storage. **CRUD** is data-oriented and the standardized use of **HTTP methods** or there is a term for this called **HTTP action verbs**. **HTTP** has a few **action verbs** or methods that work as CRUD operations.

what are standard CRUD Operations:

- **POST**: Creates a new resource
- **GET**: Reads/Retrieve a resource
- **PUT**: Updates an existing resource
- **DELETE**: Deletes a resource

Database Layer

As the name suggests,

- **CREATE Operation**: Performs the INSERT statement to create a new record.
- **READ Operation**: Reads table records based on the input parameter.
- **UPDATE Operation**: Executes an update statement on the table. It is based on the input parameter.
- **DELETE Operation**: Deletes a specified row in the table. It is also based on the input parameter.

Data Access/Integration

- The *Data Access/Integration* layer consists of the JDBC, ORM, OXM, JMS and Transaction modules.
- The JDBC module provides a JDBC-abstraction layer that removes the need to do tedious JDBC coding and parsing of database-vendor specific error codes.
- The *ORM* module provides integration layers for popular object-relational mapping APIs, including JPA, JDO, and Hibernate. Using the ORM package you can use all of these O/R-mapping frameworks in combination with all of the other features Spring offers, such as the simple declarative transaction management feature mentioned previously.
- The OXM module provides an abstraction layer that supports Object/XML mapping implementations for JAXB, Castor, XMLBeans, JiBX and XStream.
- The Java Messaging Service (JMS) module contains features for producing and consuming messages.
- The Transaction module supports programmatic and declarative transaction management for classes that implement special interfaces and for *all your POJOs (plain old Java objects)*.

Choosing a Database

- The Spring Framework provides extensive support for working with SQL databases, from direct JDBC access using JdbcTemplate to complete “object relational mapping” technologies such as Hibernate.
- Spring Data provides an additional level of functionality: creating Repository implementations directly from interfaces and using conventions to generate queries from your method names.

Configure a DataSource

- Java’s javax.sql.DataSource interface provides a standard method of working with database connections. Traditionally, a 'DataSource' uses a URL along with some credentials to establish a database connection.

1. Embedded Database Support



- It is often convenient to develop applications by using an in-memory embedded database. Obviously, in-memory databases do not provide persistent storage.
- Spring Boot can auto-configure embedded H2, HSQL, and Derby databases. You need not provide any connection URLs. You need only include a build dependency to the embedded database that you want to use.

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
    <dependency> <groupId>org.hsqldb</groupId>
    <artifactId>hsqldb</artifactId> <scope>runtime</scope>
</dependency>
```

2.Connection to a Production Database

- Production database connections can also be auto-configured by using a pooling DataSource. Spring Boot uses the following algorithm for choosing a specific implementation:
- We prefer HikariCP for its performance and concurrency. If HikariCP is available, we always choose it.
- Otherwise, if the Tomcat pooling DataSource is available, we use it.
- If neither HikariCP nor the Tomcat pooling datasource are available and if Commons DBCP2 is available, we use it.
- If you use the spring-boot-starter-jdbc or spring-boot-starter-data-jpa "starters", you automatically get a dependency to HikariCP.

2. Connection to a Production Database

- DataSource configuration is controlled by external configuration properties in spring.datasource.*.
- For example, you might declare the following section in application.properties:

```
spring.datasource.url=jdbc:mysql://localhost/test
spring.datasource.username=dbuser
spring.datasource.password=dbpass
 spring.datasource.driver-class-name=com.mysql.jdbc.Driver
```

3.Connection to a JNDI DataSource

- If you deploy your Spring Boot application to an Application Server, you might want to configure and manage your DataSource by using your Application Server's built-in features and access it by using JNDI(Java Naming and Directory Interface)
- The spring.datasource.jndi-name property can be used as an alternative to the spring.datasource.url, spring.datasource.username, and spring.datasource.password properties to access the DataSource from a specific JNDI location. For example, the following section in application.properties shows how you can access a JBoss AS defined DataSource:

```
spring.datasource.jndi-name=java:jboss/datasources/customers
```

Spring Boot Data JPA

- The Java Persistence API is a standard technology that lets you “map” objects to relational databases. The spring-boot-starter-data-jpa POM provides a quick way to get started. It provides the following key dependencies:

- Hibernate: One of the most popular JPA implementations.
- Spring Data JPA: Helps you to implement JPA-based repositories.
- Spring ORM: Core ORM support from the Spring Framework.

Entity Classes

- Traditionally, JPA “Entity” classes are specified in a persistence.xml file. With Spring Boot, this file is not necessary and “Entity Scanning” is used instead. By default, all packages below your main configuration class (the one annotated with @EnableAutoConfiguration or @SpringBootApplication) are searched.

- Any classes annotated with @Entity, @Embeddable, or @MappedSuperclass are considered. A typical entity class resembles the following example:

Entity Classes

```
package com.example.myapp.domain;
import java.io.Serializable;
 import javax.persistence.*;
@Entity
public class City implements Serializable {
 @Id
@GeneratedValue
private Long id;
@Column(nullable = false)
private String name;
 @Column(nullable = false)
private String state; // ... additional members, often include @OneToMany mappings
protected City() {
 // no-args constructor required by JPA spec
 // this one is protected since it shouldn't be used directly
 }
 public City(String name, String state) {
this.name = name;
this.state = state;
 }
public String getName()
{
return this.name;
}
public String getState()
{
 return this.state;
} // ... etc }
```

Spring Data JPA Repositories

- Spring Data JPA repositories are interfaces that you can define to access data. JPA queries are created automatically from your method names. For example, a CityRepository interface might declare a findAllByState(String state) method to find all the cities in a given state.

- For more complex queries, you can annotate your method with Spring Data's Query annotation.

- Spring Data repositories usually extend from the Repository or CrudRepository interfaces. If you use auto-configuration, repositories are searched from the package containing your main configuration class (the one annotated with @EnableAutoConfiguration or @SpringBootApplication) down.

Spring Data JPA Repositories

The following example shows a typical Spring Data repository interface definition:

```
package com.example.myapp.domain;
import org.springframework.data.domain.*;
import org.springframework.data.repository.*;
 public interface CityRepository extends Repository<City, Long>
{
Page<City> findAll(Pageable pageable);
City findByNameAndStateAllIgnoringCase(String name, String state);
 }
```

Creating and Dropping JPA Databases

- By default, JPA databases are automatically created only if you use an embedded database (H2, HSQL, or Derby). You can explicitly configure JPA settings by using spring.jpa.* properties. For example, to create and drop tables you can add the following line to your application.properties:

```
spring.jpa.hibernate.ddl-auto=create-drop
```

```
spring.jpa.properties.hibernate.globally_quoted_identifiers=true
```

- The line in the preceding example passes a value of true for the hibernate.globally_quoted_identifiers property to the Hibernate entity manager.
- By default, the DDL execution (or validation) is deferred until the ApplicationContext has started. There is also a spring.jpa.generate-ddl flag, but it is not used if Hibernate auto-configuration is active, because the ddl-auto settings are more fine-grained.

Storing Users and Attempts

- **Storage Mechanisms**
- Each of the supported mechanisms for reading a username and password can use any of the supported storage mechanisms:
- Simple Storage with In-Memory Authentication
- Relational Databases with JDBC Authentication
- Custom data stores with UserDetailsService
- LDAP storage with LDAP Authentication

Storing Users and Attempts

- Brute force attacks are a common threat where an attacker tries multiple combinations of usernames and passwords to gain authorized access to the system. To protect against such attacks, it is crucial to implement mechanisms that can detect and block repeated failed login attempts.
- Spring Security can provide robust support to help mitigate these threats. In this article, we will learn **how to prevent brute-force authentication attempts using Spring Security.**
- The main idea is to track the number of failed login attempts and lock the user account temporarily after the predefined number of unsuccessful attempts. We will use the in-memory approach for simplicity but it can extended to use the database for better scalability of the application.

Create the SecurityConfig class

```java
package com.demo.securitypreventbruteforce.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    private final CustomAuthenticationProvider authenticationProvider;

    public SecurityConfig(CustomAuthenticationProvider authenticationProvider) {
        this.authenticationProvider = authenticationProvider;
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
                .authorizeRequests(authorizeRequests ->
                        authorizeRequests
                                .requestMatchers("/login", "/blocked").permitAll() // Allow access to login and
blocked pages
                                .anyRequest().authenticated() // All other requests require authentication
                )
                .formLogin(formLogin ->
                        formLogin
                                .loginPage("/login").permitAll() // Use custom login page
                                .defaultSuccessUrl("/home", true) // Redirect to home on successful login
                )
                .logout(logout ->
                        logout.permitAll() // Allow logout for all users
                )
                .authenticationProvider(authenticationProvider); // Use custom authentication provider

        return http.build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(); // Use BCrypt for password encoding
    }

    @Bean
    public UserDetailsService userDetailsService() {
        var userDetailsService = new InMemoryUserDetailsManager();
        var user = User.withUsername("user")
                .password(passwordEncoder().encode("password"))
                .roles("USER")
                .build();
        userDetailsService.createUser(user);
        return userDetailsService; // Set up in-memory user details service
    }
}
```

Create the CustomAuthenticationProvider class

```java
package com.demo.securitypreventbruteforce.config;

import com.demo.securitypreventbruteforce.service.BruteForceProtectionService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.authentication.BadCredentialsException;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.AuthenticationException;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.authentication.AuthenticationProvider;
import org.springframework.stereotype.Component;

@Component
public class CustomAuthenticationProvider implements AuthenticationProvider {

    @Autowired
    private UserDetailsService userDetailsService;

    @Autowired
    private PasswordEncoder passwordEncoder;

    @Autowired
    private BruteForceProtectionService bruteForceProtectionService;

    @Override
    public Authentication authenticate(Authentication authentication) throws AuthenticationException {
        String username = authentication.getName();
        String password = (String) authentication.getCredentials();

        // Check if the user is blocked due to too many failed login attempts
        if (bruteForceProtectionService.isBlocked(username)) {
            throw new BadCredentialsException("You have been temporarily locked due to too many failed login
attempts.");
        }

        User user = (User) userDetailsService.loadUserByUsername(username);

        // Verify user credentials
        if (user == null || !passwordEncoder.matches(password, user.getPassword())) {
            bruteForceProtectionService.loginFailed(username); // Record failed login attempt
            throw new BadCredentialsException("Invalid username or password.");
        }

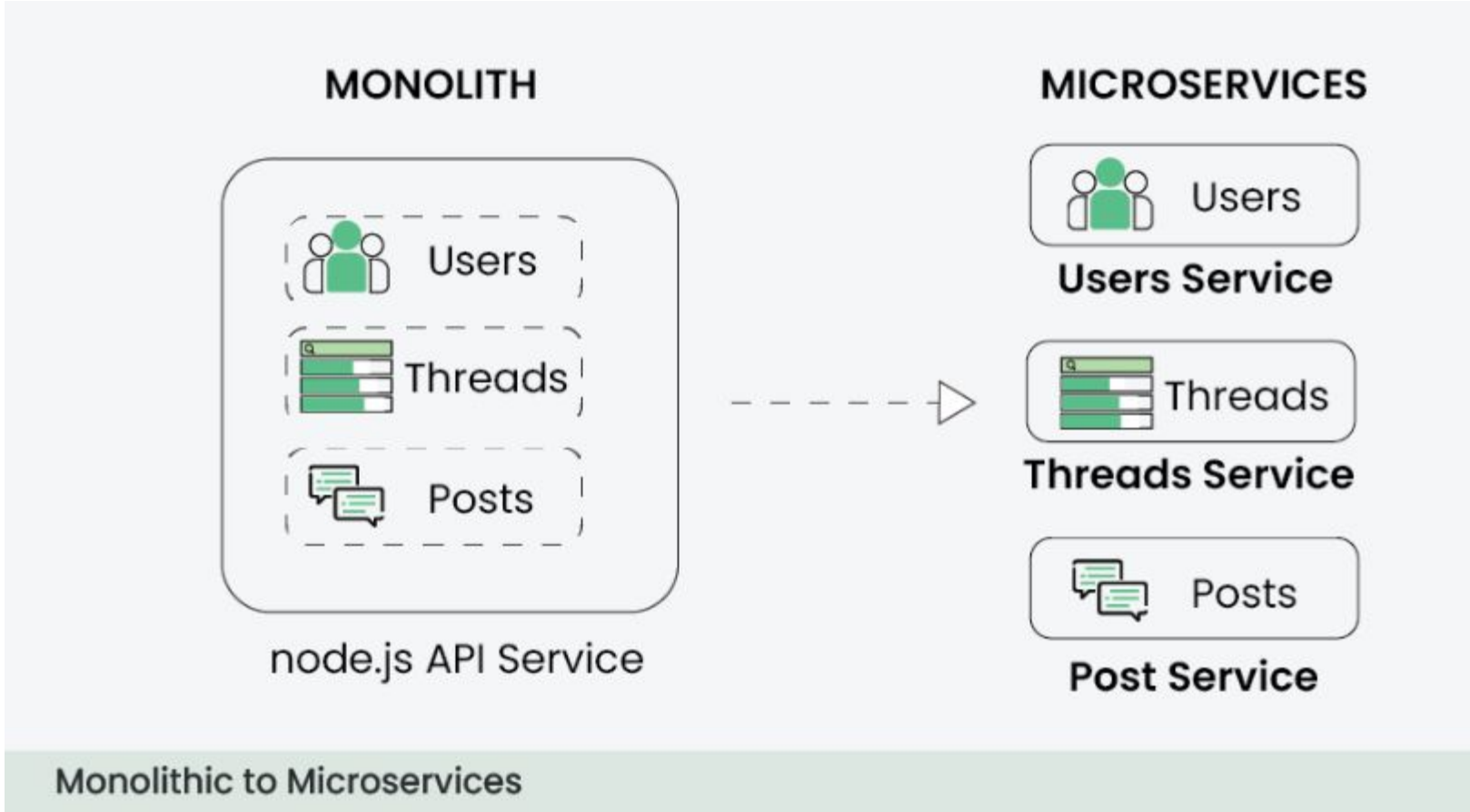
        bruteForceProtectionService.loginSucceeded(username); // Record successful login
        return new UsernamePasswordAuthenticationToken(username, password, user.getAuthorities());
    }

    @Override
    public boolean supports(Class<?> authentication) {
        return UsernamePasswordAuthenticationToken.class.isAssignableFrom(authentication);
    }
}
```


MICROSERVICES

MICROSERVICES

- Microservices are an architectural approach to develop software applications as a collection of **small, independent services** that communicate with each other over a network.
- Instead of building a monolithic application where **all the functionality is tightly integrated into a single codebase**, microservices break down the application into smaller, loosely coupled services.
- This architecture allow you to take a **large monolith application and decompose it into small manageable components/services**. Also, it is considered as the building block of modern applications.
- Microservices can be written in a variety of programming languages, and frameworks, and each service acts as a mini-application on its own.
-

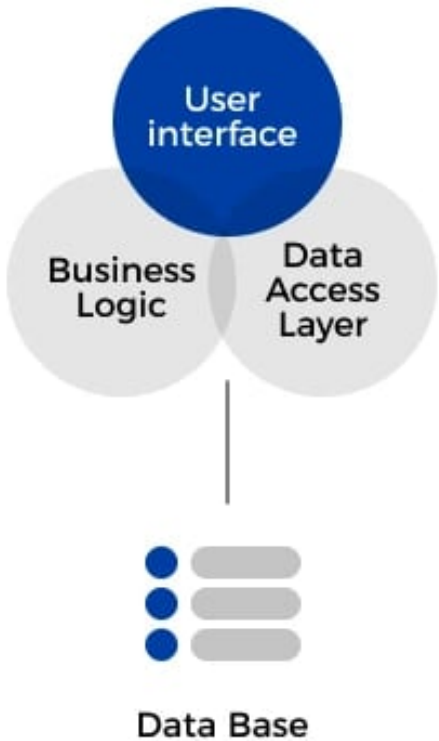


Monolithic to Microservices

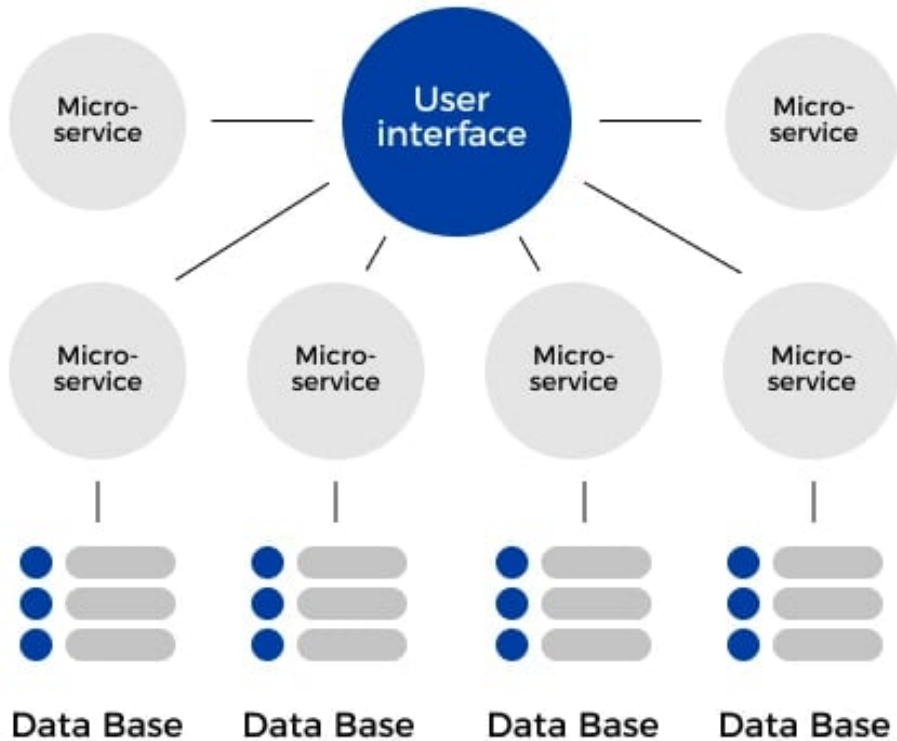
Monolithic Approach vs. Microservices

- **Monolithic Architecture for Simplicity**: Initially, the project could be built as a monolithic architecture, where all features are packaged in a single deployable unit. This would be suitable for small teams or solo developers.
- **Simplicity in a Small Team**: For small projects, adding features like gamification in the same codebase (a single deployable unit) keeps things simple and manageable.
- **Challenges with Growth**: As the system grows, with more features and users, the limitations of a monolithic approach become clear—especially in terms of complexity in managing and deploying different parts of the application.
- **Scaling Issues**: Monolithic applications are harder to scale because the entire system must be deployed together, making it inefficient when handling larger teams and user bases.

MONOLITHIC ARCHITECTURE



MICROSERVICE ARCHITECTURE



Monolithic Architecture	Microservices Architecture
Consists of a singe codebase with multiple modules within according to the business functionalities.	Consists of individual services with each service being responsible for exactly one functionality.
Do not need expert domain knowledge for development.	Risky to implement without domain expertise and container knowledge.
Easier deployment.	Relatively complex deployment.
Updating the system is a tedious process which would need the entire system to be redeployed.	Only the service which is updated needs to be redeployed.
Reusing the modules from one software into other software systems is difficult.	Microservices can be easily used in development of other software.

Moving to Microservices

As the business grows and new features are introduced, the monolithic approach starts showing its limits. Some of the new feature ideas mentioned include:

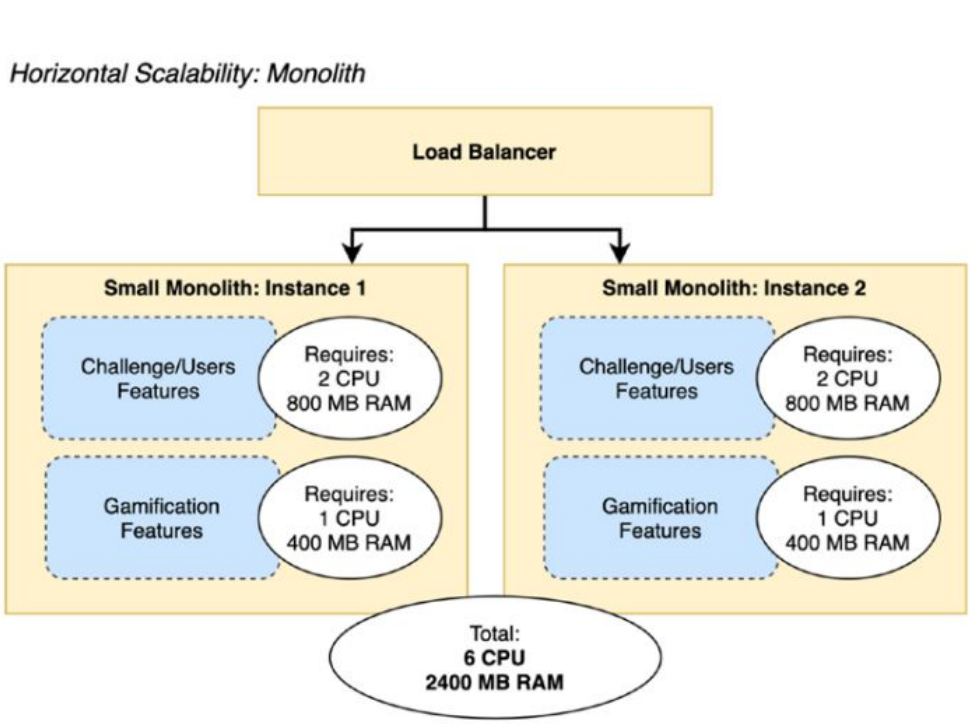
- **Adaptive Challenges**: Adjusting the complexity of challenges based on user statistics.
- **Hints**: Providing hints during challenges.
- **User Login**: Moving from anonymous users to registered users.
- **Personal Data Collection**: Collecting user data to improve metrics and insights.

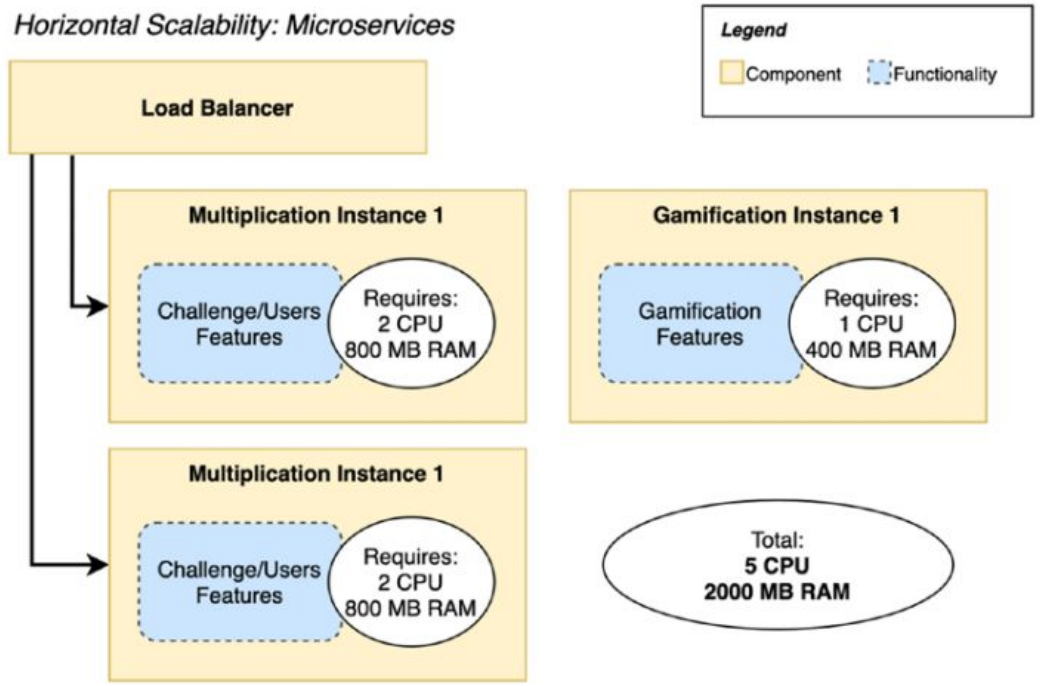
These new features would impact existing domains like **Users** and **Challenges**, making them more complex. Implementing these features within a monolith would increase the coupling between components, making development slower and more prone to issues when one component fails.

Scalability and Availability

- **Users and Challenges Domains**: These are crucial parts of the application that need to be highly available.
  - For example, if users can't log in or challenges can't be loaded, it directly affects the core functionality of the system. **To ensure high availability, a microservices architecture allows horizontal scalability—where multiple instances of the same service can be deployed across servers to handle increased traffic** and ensure reliability through load balancing. This makes the system more resilient to failures and capable of handling concurrent users more effectively.
- **Gamification Domain**: Unlike the critical domains, gamification features (like providing points or badges) are non-critical. If these features are temporarily unavailable, users can still interact with the core functionality of the app (solving challenges). Therefore, the gamification service doesn't need to be scaled or made as highly available as the other components. In a microservices architecture, this service can be handled with a lower priority for scaling.

Horizontal scalability in a monolith versus microservices





Independent Deployment and Parallel Development



- With an influx of investment and the team growing, development can no longer be sequential.
- In a monolith, when multiple teams work on different parts of the same codebase, they often step on each other's toes, especially during deployment.
- All components of the application must be deployed together, leading to long testing cycles, dependencies between teams, and risks that a bug in one part of the system will bring down the entire application.
- **HOW MICROSERVICES ADDRESSES THIS PROBLEM :**

  Microservices address this by allowing **independent deployments**. Each feature or domain, such as gamification, users, and challenges, can be developed, tested, and deployed separately. Teams can work in isolation on their respective services without worrying about breaking other parts of the application. This reduces dependencies between teams and accelerates development, as each microservice is loosely coupled and interacts with others through well-defined APIs.

Microservices Advantages

- **Independent Workflows**: Teams work on their own microservices, enabling independent changes, updates, and scaling without cross-team coordination.
- **Modular Development with DDD**: Using Domain-Driven Design, architecture is split into bounded contexts, reducing integration challenges by allowing separate repositories and deployment cycles.
- **Independent Infrastructure**: Each microservice manages its own infrastructure (e.g., database, web server), aligning with DevOps principles and simplifying maintenance.
- **Decoupling for Flexibility**: Microservices enable independent updates and replacements without affecting the entire system, unlike monolithic architectures.

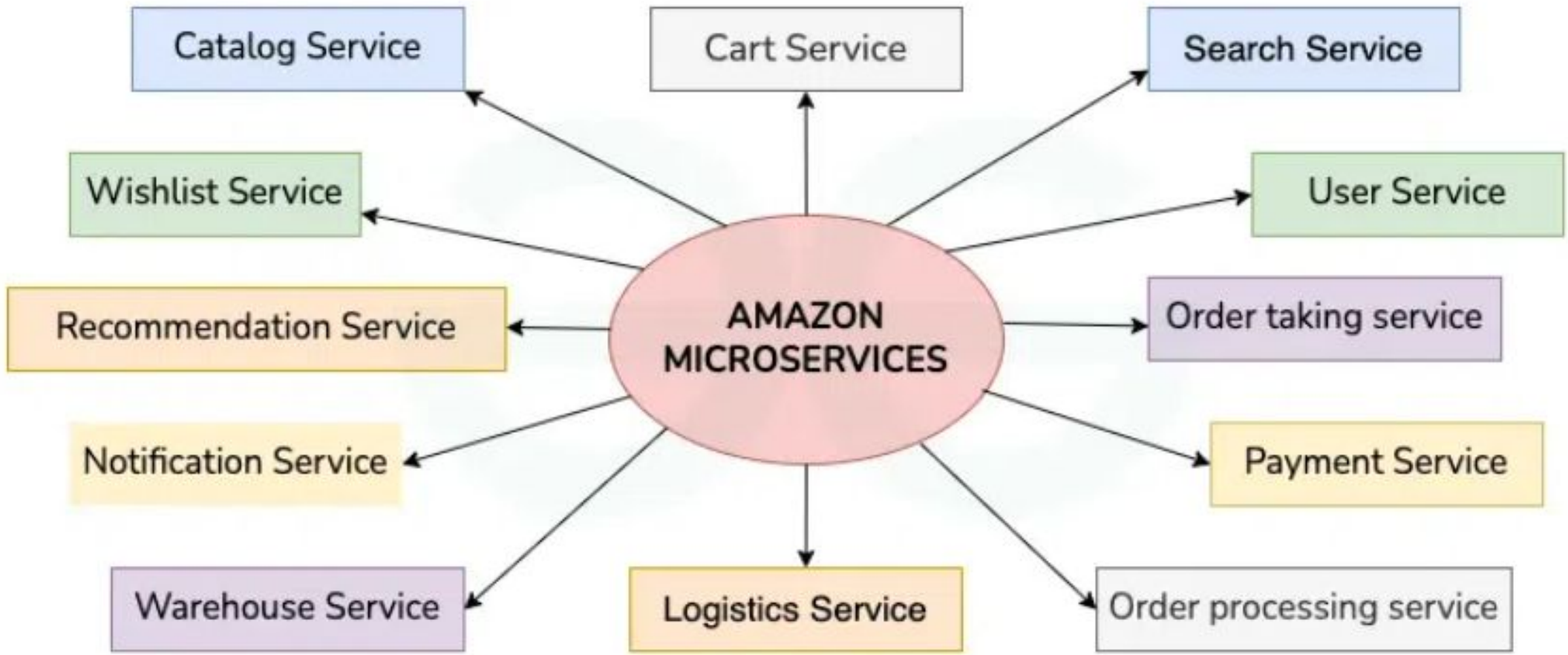
Challenges of using Microservices Architecture

- Managing service communication, network latency, and data consistency can be difficult.
- Decomposing an app into microservices adds complexity in development, testing, and deployment.
- Network communication can lead to higher latency and complicates error handling.
- Maintaining consistent data across services is challenging, and distributed transactions can be complex.

Real-World Examples of Companies using Microservices Architecture

Organizations have undergone significant changes by adopting microservices, moving from monolithic applications. Here are some real-life examples:

- **Amazon:** Initially a monolithic app, Amazon uses microservices early on, breaking its platform into smaller components. This shift allowed for individual feature updates, greatly enhancing functionality.
- **Netflix:** After facing service outages while transitioning to a movie-streaming service in 2007, Netflix adopted a microservices architecture. This change improved reliability and performance.
- **Uber:** By switching from a monolithic structure to microservices, Uber operations were become smoother, resulting in increased webpage views and search efficiency


Catalog Service
Cart Service
Search Service
Wishlist Service
User Service
Recommendation Service
AMAZON MICROSERVICES
Order taking service
Notification Service
Payment Service
Warehouse Service
Logistics Service
Order processing service

Technologies that enables Microservices Architecture

- **Docker:** A platform that packages applications and their dependencies into portable containers, ensuring they run consistently across different environments.
- **Kubernetes:** An open-source tool that automates the deployment and management of containers, handling tasks like scaling and load balancing.
- **Event-Driven Architecture:** This setup allows services to communicate by sending and receiving events, using systems like Apache Kafka for reliable messaging.
- **Serverless Computing:** Platforms like AWS Lambda let you run microservices without managing servers, simplifying deployment and scaling

Architecture Overview

- Create a new Spring Boot application for our gamification requirements. Now we can refer to these two applications as
- **Multiplication microservice**
- **Gamification microservices.**

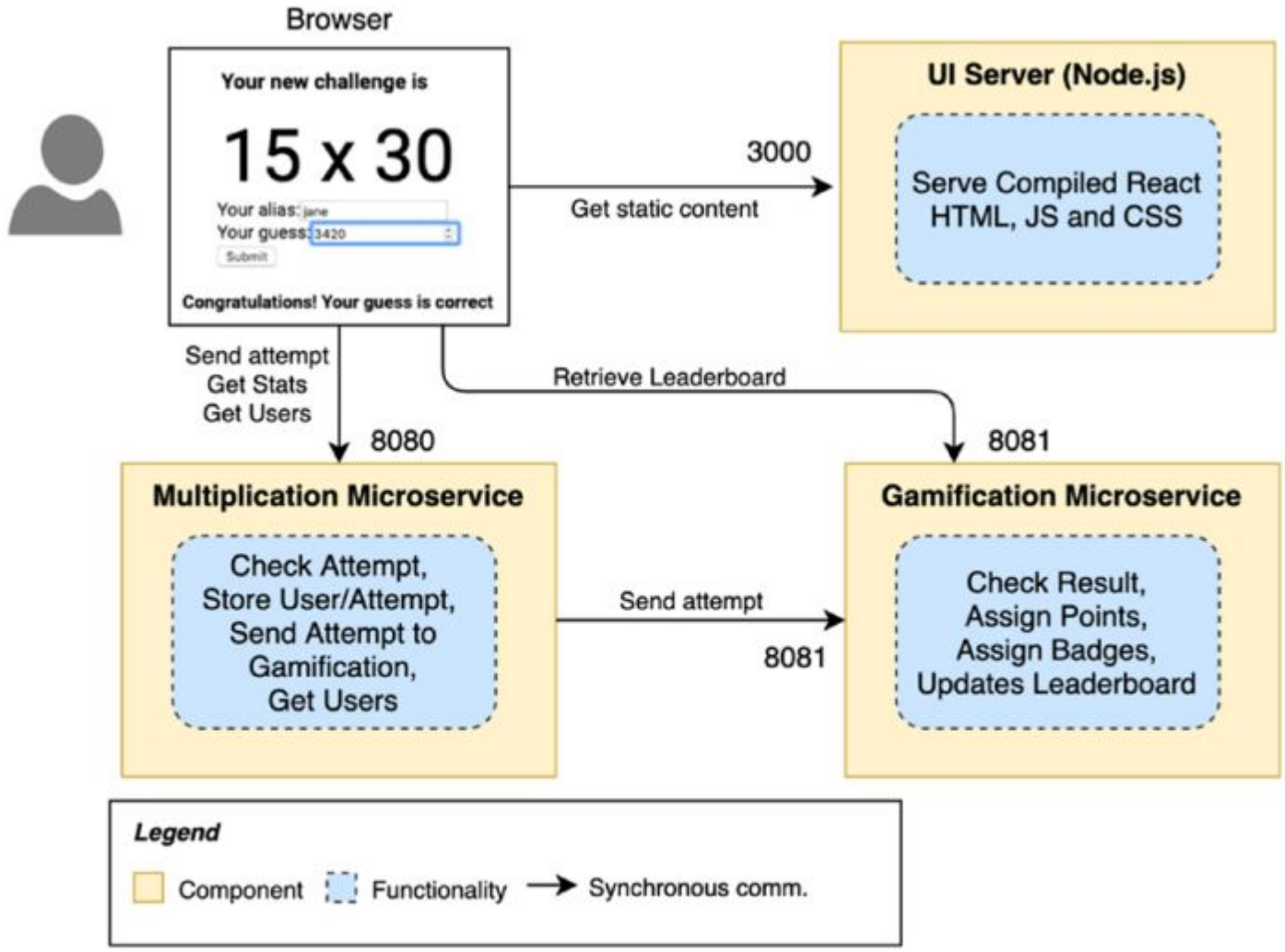


Figure 6-2. Logical view

New additions

- There will be a new microservice**, Gamification**. To prevent local port conflicts, we'll deploy it on port 8081.
- The Multiplication microservice will send each attempt to the Gamification microservice to process new score, badges, and update the leaderboard.
- There will be a new component in our React UI to render the leaderboard with the score and badges. As you can see in the drawing, our UI will call both microservices.

Designing and Implementing the New Service

we'll design and implement the Gamification microservice.

- Interfaces:

- **Importance of Interfaces**: Clear contracts between modules are essential, especially in microservices.
- **Gamification Microservice API**: It will expose a REST API to accept new attempts for user statistics.
- **Data Exchange Format**: The API will use a JSON object containing the attempt's and user's data, with only the necessary data utilized by the Gamification service.
- **Leaderboard Access**: New REST endpoints will be created in the Gamification microservice to allow the UI to collect leaderboard details.
- **Streamlined Communication**: Defining clear dependencies and interfaces facilitates efficient communication between microservices.

The Spring Boot Skeleton for Gamification



- We can use again the Spring Initializr at https://start.spring.io/ to create the basic skeleton for our new application.
- We need some extra dependencies, so we can add them directly from here: Lombok, Spring Web, Validation, Spring Data JPA, and the H2 Database.

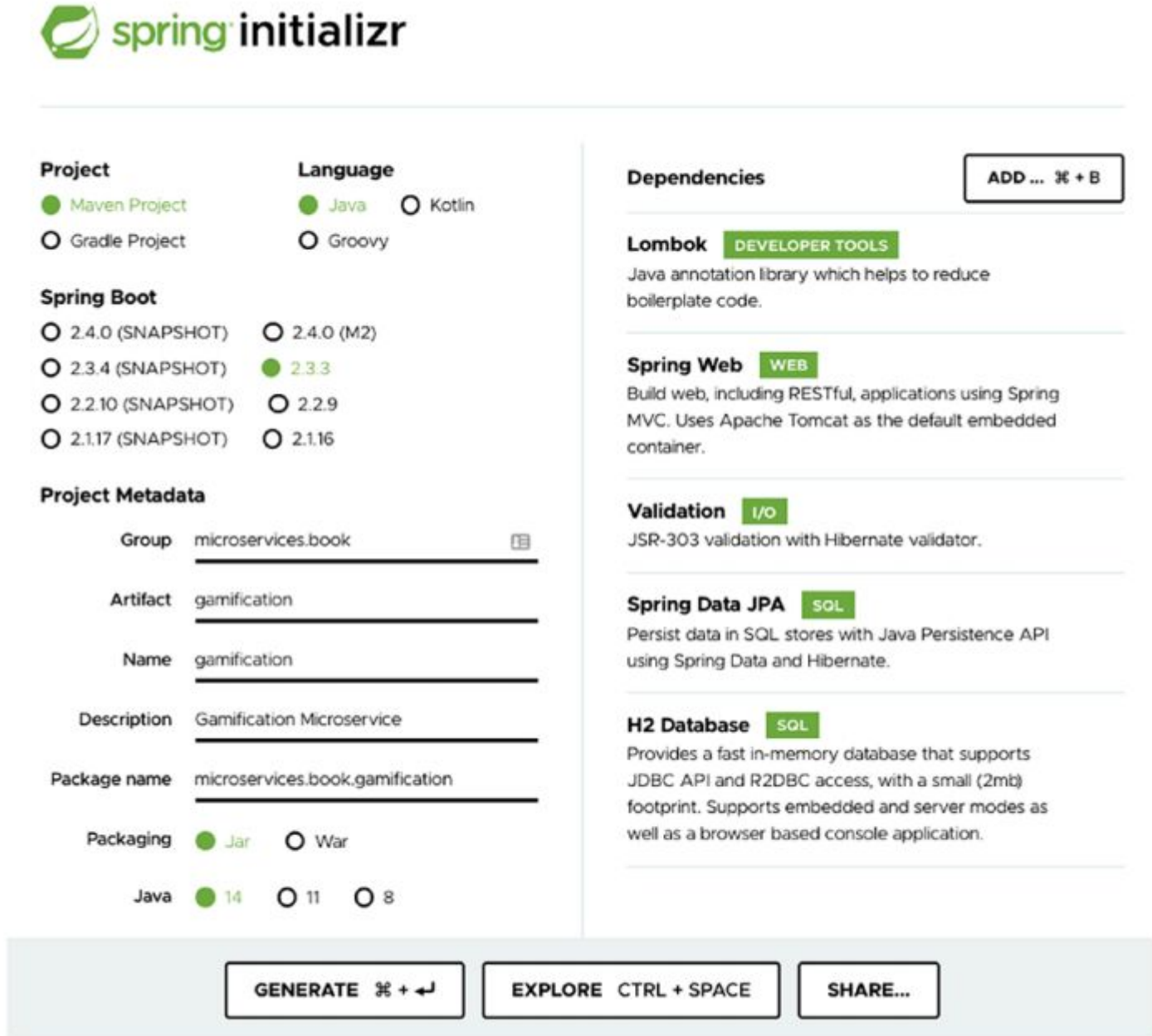


Figure 6-3. *Creating the Gamification application*

Modelling the Gamification domain

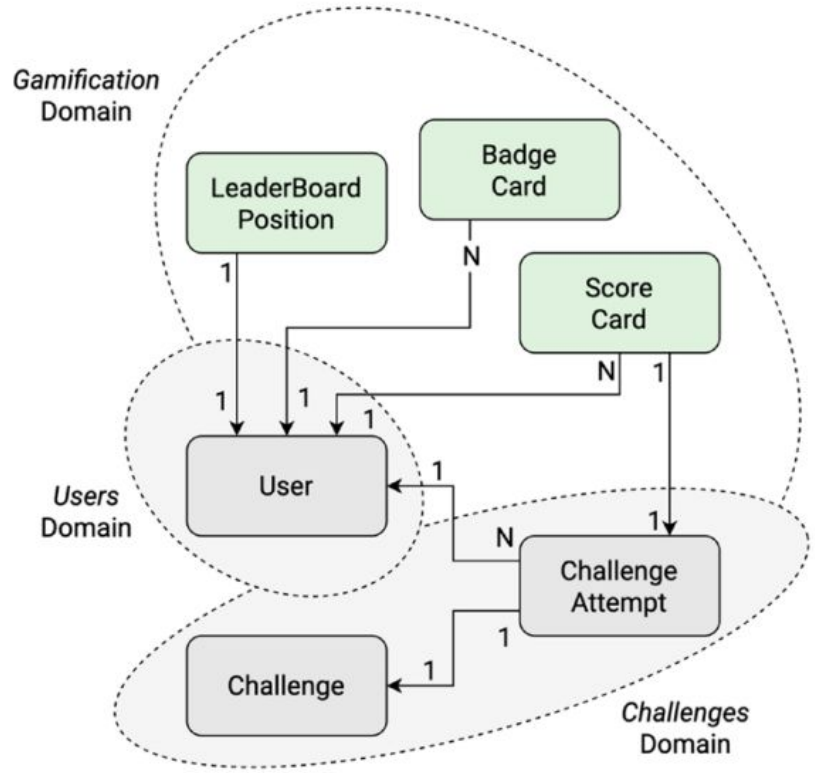


Figure 6-4. *The new Gamification domain*

Let's model our gamification domain trying to respect the context boundaries and minimizing the coupling with the existing functionality.

- We create a score card object, which holds the amount of score that a user obtains for a given challenge attempt.
- Similarly, we have a badge card object, representing a specific type of badge that has been won at a given time by a user. It doesn't need to be tied to a score card since you may win a badge when you surpass a given score threshold.
- To model the leaderboard, we create a leaderboard position. We'll display an ordered list of these domain objects to show the ranking to the users.

Source Code



You can find all the source code for this chapter on GitHub, in the chapter06 repository.

See https://github.com/Book-Microservices-v2/chapter06.

ScoreCard class, with an extra constructor

```
import lombok.*;
import javax.persistence.*;
/**
* This class represents the Score linked to an attempt in the game, * with an associated user and the timestamp
in which the score * is registered.
*/
@Entity
@Data
@AllArgsConstructor
@NoArgsConstructor
public class ScoreCard {
 // The default score assigned to this card, if not specified.
 public static final int DEFAULT_SCORE = 10;
 @Id
 @GeneratedValue
 private Long cardId;
 private Long userId;
 private Long attemptId;
 @EqualsAndHashCode.Exclude
 private long scoreTimestamp;
 private int score;
 public ScoreCard(final Long userId, final Long attemptId) {
 this(null, userId, attemptId, System.currentTimeMillis(), DEFAULT_SCORE);
 }
}
```

- We used a new Lombok annotation, @EqualsAndHashCode.Exclude.
- As the name suggests, this will make Lombok not to include that field in the generated equals and hashCode methods. The reason is that this will make our tests easier when we compare objects, and in fact, we don't need the timestamp to determine whether two cards are equal.

The BadgeType Enum

```
The BadgeType Enum
package microservices.book.gamification.game.domain;
import lombok.Getter;
import lombok.RequiredArgsConstructor;
/**
* Enumeration with the different types of Badges that a user can win.
*/
@RequiredArgsConstructor
@Getter
public enum BadgeType {
 // Badges depending on score
 BRONZE("Bronze"),
 SILVER("Silver"),
 GOLD("Gold"),
 // Other badges won for different conditions
 FIRST_WON("First time"),
 LUCKY_NUMBER("Lucky number");
 private final String description;
}
```

- The different badges are defined in an enum, BadgeType.
- We'll add a description field to have a friendly name for each one.
- In the previous code, we also benefit from some Lombok annotations in enums.
- In this case, we use them to generate a constructor and the getter for the description field.

The BadgeCard Domain/Data Class

```
package microservices.book.gamification.game.domain;
import lombok.*;
import javax.persistence.*;
@Entity
@Data
@AllArgsConstructor
@NoArgsConstructor
public class BadgeCard {
 @Id
 @GeneratedValue
 private Long badgeId;
 private Long userId;
 @EqualsAndHashCode.Exclude
 private long badgeTimestamp;
 private BadgeType badgeType;
 public BadgeCard(final Long userId, final BadgeType badgeType) {
 this(null, userId, System.currentTimeMillis(), badgeType);
 }
}
```

- The BadgeCard class uses BadgeType, and it's also a JPA entity.
- Added a constructor to set some default values. Note that we don't need to add any specific JPA annotation to the enum type.
- Hibernate will map the values to the enum's ordinal value (an integer) by default. This works just fine if we keep in mind that we should only append new enum values at the end, but we could also configure the mapper to use the string values.

The LeaderBoardRow Class

Listing 6-4. The LeaderBoardRow Class

```
package microservices.book.gamification.game.domain;

import lombok.*;
import java.util.List;

@Value
@AllArgsConstructor
public class LeaderBoardRow {

    Long userId;
    Long totalScore;
    @With
    List<String> badges;

    public LeaderBoardRow(final Long userId, final Long totalScore) {
        this.userId = userId;
        this.totalScore = totalScore;
        this.badges = List.of();
    }

}
```

- To model the leaderboard position, we create the class LeaderBoardRow.
- We don’t need to persist this object in our database since it’s going to be created on the fly by aggregating scores and badges from our user.
- The @With annotation added to the badges field is provided by Lombok and generates a method for us to clone an object and add a new field value to the copy (in this case, withBadges).
- This is a good practice when we work with immutable classes since they don’t have setters. We’ll use this method when we create the business logic to merge the score and the badges for each leaderboard row.

Service

The business logic in this new Gamification microservice is divided into two.

- The game logic, responsible for processing the attempt and generating the resulting score and badges
- The leaderboard logic, which aggregates data and builds the ranking based on score

- The game logic will reside in the class GameServiceImpl, which implements the GameService interface. The specification is simple: based on an attempt, it computes the score and badges and stores them.
- This business logic is accessible to the Multiplication microservice via a controller named GameController, which will expose a POST endpoint to send the attempt to. On the persistence layer, our business logic will require a ScoreRepository to save scorecards and a BadgeRepository to do the same thing with badge cards.

UML diagram with all the classes needed to build the game logic functionality

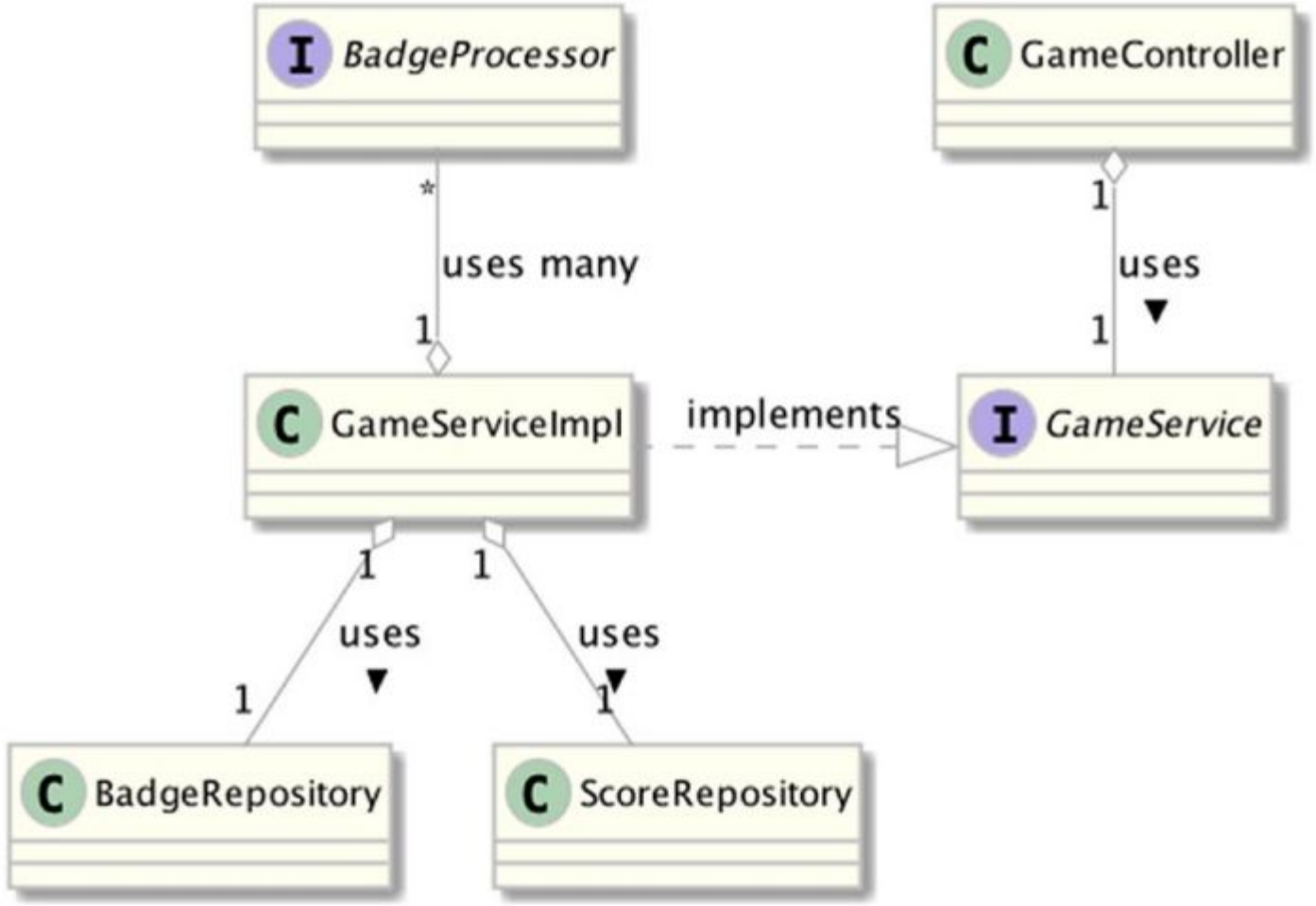


Figure 6-5. *UML: game logic*

Kindly refer TEXTBOOK for the remaining source code

Full source code from page number : 179 -206

User Interface (UI)

The back-end logic is ready, so we can move to the front-end part. We need two new

❑ JavaScript classes:

- A new API client to retrieve the leaderboard data from the Gamification microservice
- An additional React component to render the leaderboard.

We'll also add a new method to the existing API client to retrieve a list of users based on their IDs.

Listing 6-27. The GamiApiClient Class

```
class GameApiClient {
    static SERVER_URL = 'http://localhost:8081';
    static GET_LEADERBOARD = '/leaders';

    static leaderBoard(): Promise<Response> {
        return fetch(GameApiClient.SERVER_URL +
            GameApiClient.GET_LEADERBOARD);
    }

}

export default GameApiClient;
```

Listing 6-28. Renaming the Former Apiclient Class and Including the New Call

```
class ChallengesApiClient {

    static SERVER_URL = 'http://localhost:8080';
    // ...
    static GET_USERS_BY_IDS = '/users';

    // existing methods...

    static getUsers(userIds: number[]): Promise<Response> {
        return fetch(ChallengesApiClient.SERVER_URL +
            ChallengesApiClient.GET_USERS_BY_IDS +
            '/' + userIds.join(','));
    }
}

export default ChallengesApiClient;
```

- The GameApiClient class in Listing 6-27 defines a different host and uses the fetch API to retrieve the JSON array of objects.
- For clarity, we also rename the existing ApiClient to ChallengesApiClient.
- Then, we include in this one a new method to retrieve the users. See Listing 6-28.

User Interface (Contd…)

Listing 6-29. The New LeaderBoardComponent in React

Listing 6-30. Adding the Badge Style to App.css

Listing 6-31. Adding the LeaderBoardComponent Inside the ChallengeComponent

Kindly refer TEXT BOOK [E-book – page nums: 208-212] for the above codes.

Playing with the System

- The new Gamification microservice has been implemented, connected the Multiplication application to it via a REST API client service, and built the UI to fetch the leaderboard and render it every five seconds.
- It's time to play with our complete system. Use your IDE or the command line to start both back-end applications and the Node.js server. If you use the terminal, open three separate instances and run one of the commands in Listing 6-32 in each of them so you have access to all the logs separately.

Playing with the System

Listing 6-32. Starting the Apps from the Console

```
/multiplication $ mvnw spring-boot:run
...
/gamification $ mvnw spring-boot:run
...
/challenges-frontend $ npm start
...
```

OUTPUT:

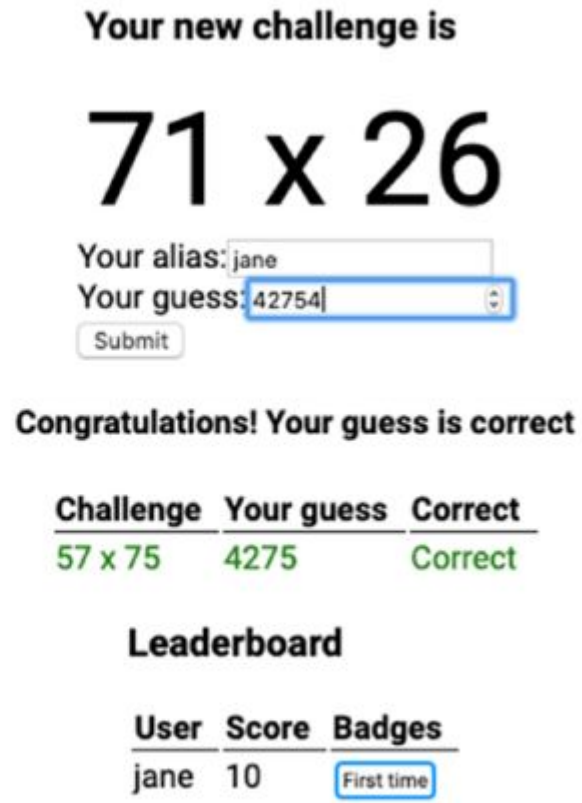


Figure 6-7. *UI connected to both microservices*

The moment we send a first correct attempt, we'll get 10 points and the "First time" badge.

The game works! You can keep playing to see whether you score any metal badges or the Lucky number one. Thanks to the automatic rendering every five seconds, you can even play in multiple browser tabs and the leaderboard will be refreshed in each one of them.